



TEMA 2 – INSTALACIÓN Y CONFIGURACIÓN DE TAILWIND CSS 4

Objetivos del tema

- Entender las distintas formas de usar Tailwind
- Instalar Tailwind CSS 4 en un proyecto
- Reconocer la estructura básica de un proyecto
- Configurar Tailwind para empezar a trabajar

Formas de usar Tailwind CSS

Existen **dos formas principales** de usar Tailwind:

1 Uso rápido (CDN)

- Ideal para aprender y probar
- No requiere instalación
- No recomendado para producción

2 Instalación profesional

- Requiere Node y npm
- Genera CSS optimizado
- Forma recomendada en proyectos reales

Ejercicio 1

Crea un archivo `index.html` y añade Tailwind usando CDN.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Tailwind CDN</title>
  <script src="https://cdn.tailwindcss.com"></script>
</head>
<body>

  <h1 class="text-3xl text-blue-600">
    Tailwind funciona
  </h1>

</body>
</html>
```

👉 Comprueba que el texto se ve grande y azul.

Limitaciones del uso con CDN

Usar Tailwind por CDN:

- ✗ No permite personalización avanzada
- ✗ CSS muy grande
- ✗ No es óptimo para producción

✓ Útil solo para:

- Aprender
- Prototipos rápidos
- Ejercicios en clase

Requisitos para instalación profesional

Para instalar Tailwind necesitamos:

- Node.js
- npm (incluido con Node)
- Editor de código (VS Code recomendado)
- Terminal / consola

 Tailwind se integra en herramientas modernas

Ejercicio 2

Instala NPM. Abre la terminal y ejecuta:

```
node -v
```

```
npm -v
```

👉 Si aparece un número, está correctamente instalado.

Creando un proyecto básico

Pasos generales:

1. Crear carpeta del proyecto
2. Inicializar npm
3. Instalar Tailwind CSS
4. Crear archivos necesarios

Comando inicial:

```
npm init -y
```

📌 Se genera el archivo `package.json`

Ejercicio 3

1. Crea una carpeta llamada `tailwind-proyecto`
2. Entra en la carpeta
3. Ejecuta:

```
npm init -y
```

👉 Comprueba que aparece `package.json`

Instalación de Tailwind CSS 4

Instalamos Tailwind como dependencia:

```
npm install tailwindcss @tailwindcss/postcss postcss
```

- ✓ Tailwind CSS
- ✓ PostCSS
- ✓ Integración moderna

Ejercicio 4

Ejecuta el comando de instalación: `npm install tailwindcss @tailwindcss/postcss postcss`

👉 Al finalizar:

- Aparece la carpeta `node_modules`
- Se actualiza `package.json`

Archivos principales del proyecto

Después de instalar Tailwind tendremos:

- `package.json`
- `node_modules/`
- Archivo CSS principal que habrá que crear

📌 Tailwind se procesa antes de llegar al navegador

Documentación: <https://tailwindcss.com/docs/installation/tailwind-cli>

Archivo CSS principal

Creamos un archivo `src/input.css` con:

```
@import "tailwindcss";
```

 Estas directivas cargan todo Tailwind

Ejercicio 5

1. Crea el archivo `input.css`
2. Copia el código anterior

👉 Este archivo será la base de estilos

Generando el CSS final

Comando para generar el CSS:

```
npx @tailwindcss/cli -i input.css -o output.css --watch
```

- ✓ Convierte Tailwind en CSS real
- ✓ Modo automático (`--watch`)

Personalizando tailwind

Comando para generar el CSS
hay que crear tailwind.config.js

Sirve para **personalizar Tailwind** sin tocar sus archivos internos.

Todo lo que pongas aquí **sobrescribe o extiende el tema por defecto**, y también configura **qué archivos escanear para purgar clases** en producción.

1. **Indicar qué archivos usar para generar CSS mínimo (content/purge)**

```
export default = {  
  content: [  
    "./src/**/*.html",  
    "./src/**/*.js",  
    "./src/**/*.vue",  
  ],  
}
```

2. **Extender o sobrescribir el tema** (colores, fuentes, tamaños, etc.

```
export default = {  
  theme: {  
    extend: {  
      colors: {  
        brand: '#1c64f2',  
      },  
      spacing: {  
        128: '32rem',  
      },  
    },  
  },  
}
```

4. **Activar Dark Mode:**

```
export default = {  
  darkMode: 'class', // o 'media'  
}
```

3. **Configurar plugins** (formularios, tipografía, etc.):

```
export default = {  
  plugins: [  
  
    require('@tailwindcss/forms'),  
  
    require('@tailwindcss/typography'),  
  ],  
}
```

Ejercicio 6

Crea la configuración mínima de tailwind:

```
export default {
  content: ["*.html"],
  theme: {
    extend: {},
  },
}

<!doctype html>
<html>
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <link href="output.css" rel="stylesheet">
</head>
<body>
  <h1 class="text-3xl font-bold underline">
    Hello world!
  </h1>
</body>
</html>
```

Ejercicio 7

Ejecuta el comando de generación. `npx @tailwindcss/cli -i input.css -o output.css --watch`

👉 Comprueba que:

- Aparece `output.css` y incluye las clases que has usado en el `index.html`
- Actualiza el HTML modificando el html o añadiendo nuevos usos de elementos tailwind para que veas que en el `output.css` se muestran las clases que acabas de definir y que en la consola se ve la actualización.

Conectando HTML con Tailwind

Enlazamos el CSS generado:

```
<link href="output.css" rel="stylesheet">
```

Ejemplo completo:

```
<h1 class="text-4xl text-red-600">  
  Tailwind instalado correctamente  
</h1>
```

Flujo de trabajo con Tailwind

1. Escribes HTML con clases
2. Tailwind genera CSS
3. El navegador carga CSS optimizado



Solo se incluye lo que realmente usas

Errores comunes

- No ejecutar `--watch`
- Ruta incorrecta al CSS
- No refrescar el navegador
- Errores de escritura en clases

Resumen

- Tailwind puede usarse con CDN o instalación
- La instalación profesional es la recomendada
- Tailwind genera CSS optimizado
- Ya tenemos el entorno listo para diseñar

Tarea final

1. Crea un `index.html`
2. Enlaza `output.css`
3. Añade un texto con:
 - Color
 - Tamaño grande
 - Padding

👉 Si funciona, la instalación es correcta.